

CZOI Toolkit — Technical Specification

Version 1.0.0 · Reference specification for the Constrained Zoned-Object Implementation

Table of Contents

1. [Scope](#)
 2. [Normative References](#)
 3. [Terminology and Definitions](#)
 4. [Conformance](#)
 5. [Architecture Overview](#)
 6. [Formal Model](#)
 7. [Module Specifications](#)
 8. [Interface Specifications](#)
 9. [Semantic Rules](#)
 10. [Persistence and Serialisation](#)
 11. [Thread Safety and Concurrency](#)
 12. [Error Handling](#)
 13. [Performance Characteristics](#)
 14. [Security Considerations](#)
 15. [Versioning and Compatibility](#)
 16. [Compliance Matrix](#)
 17. [Appendices](#)
-

1. Scope

1.1 Purpose

This specification defines the CZOI toolkit — a reference implementation of the Constrained Zoned-Object Architecture (CZOA) as described in Wang (2026). It specifies:

- The **data model** — classes, attributes, invariants, and lifecycles.
- The **behaviour** — permission decisions, constraint evaluation, daemon execution.
- The **interfaces** — public APIs, protocols, and extension points.
- The **semantics** — how the toolkit realises the formal CZOA theory.

1.2 Intended audience

- **Integrators** — engineers embedding CZOI into production systems.
- **Verifiers** — researchers checking conformance to the CZOA theory.
- **Extenders** — developers adding new modules, adapters, or neural components.
- **Auditors** — reviewers checking that a CZOI deployment meets its stated guarantees.

1.3 Out of scope

The following are explicitly **not** covered by this specification:

- Deployment topology (see the User Guide §14).
- Persistence technology choice (see §10 of this spec).
- Web framework integration specifics (see the toolkit's integration modules).
- Neural network architectures beyond the built-in primitives (see §7.8).

1.4 Document conventions

- **MUST, MUST NOT, SHOULD, SHOULD NOT, MAY** are used per RFC 2119.
- Type annotations follow PEP 484.
- Exceptions are specified by class name; the exception hierarchy is defined in §12.
- Method signatures use Python 3.9+ syntax.

2. Normative References

Reference	Description
[CZOA-2026]	Wang, H. <i>Constrained Zoned-Object Architecture (CZOA): A Unified Framework for Building Secure and Intelligent Integrated Organizational Systems</i> . Preprint, 2026.
[COH-2025]	Wang, H. <i>Constrained Object Hierarchies as a Unified Theoretical Model for Intelligence and Intelligent Systems</i> . Computers, 14(11):478, 2025.
[ZRB-2026]	Wang, H. <i>A Formalized Zoned Role-Based Framework</i> . Computers, 15(3):1–26, 2026.
[UNILOG-2026]	Wang, H. <i>UniLog: A Unified Logic Framework for Constrained Object Hierarchies and Constrained Zone-Object Architecture</i> . Preprint, 2026.
[RFC-2119]	Bradner, S. <i>Key words for use in RFCs to Indicate Requirement Levels</i> . IETF, 1997.
[PEP-484]	van Rossum, G., Lehtosalo, J., Langa, Ł. <i>Type Hints</i> . Python Enhancement Proposal, 2014.
[NIST-SP-800-162]	Hu, V. et al. <i>Guide to Attribute Based Access Control (ABAC) Definition and Considerations</i> . NIST, 2014.

3. Terminology and Definitions

3.1 Core terms

Zone — A recursive organisational unit. Every zone is a complete CZOI system and may contain child zones. Formally, $z \in Z$ where Z is the zone set of its parent.

AtomicZone — A zone with no children. Formally, z where $z.zones = \emptyset$.

CompositeZone — A zone that may contain children. Formally, z where $z.zones$ is a finite (possibly empty) mapping.

Role — A named job function defined in a specific zone. Formally, $r = (name, zone, P_base, junior_roles, properties)$.

User — An authenticated identity. Formally, $u = (\text{name}, \text{roles}, \text{attributes}, \text{credentials})$ where **roles** is a set of role names.

Application — A structural module grouping operations. Applications are **not** permission targets.

Operation — An atomic executable action. Operations are the **only** permission targets in CZOI.

Effective permission set — The union of a role's base permissions and those of all transitive junior roles. Formally, $P_{\text{eff}}(r) = P_{\text{base}}(r) \cup \bigcup_{\{r' \in \text{junior}^*(r)\}} P_{\text{base}}(r')$.

Decision — The result of a permission check. One of **ALLOW**, **DENY**, or **INCONCLUSIVE**.

Constraint — A formal rule expressed in UniLang. Four families: I (identity), T (trigger), G (goal), C (access).

Daemon — A continuous monitoring process. Emits typed signals up a daemon tree.

Signal — A typed message emitted by a daemon. One of **STATE_NORMAL**, **STATE_WARNING**, **STATE_CRITICAL**, **REVOKE**, **ESCALATE**.

Neural component — A learnable function attached to a zone. Any object with a **predict** method.

Embedding — A vector representation of an entity in a Hilbert space.

γ (gamma) mapping — An inter-zone role relation that transfers permissions from a parent role to a child role.

3.2 Notation

Symbol	Meaning
S	A CZOI system (10-tuple)
z	A zone
Z	The zone set
R	The role set
U	The user set
A	The application set
O	The operation set
N	The neural component set
E	The embedding function
Γ	The constraint system
Φ	The permission calculus
Δ	The daemon set
$P_{\text{base}}(r)$	The base permission set of role r
$P_{\text{eff}}(r, z)$	The effective permission set of role r in zone z

Symbol	Meaning
U_z	Users of zone z
R_z	Roles of zone z
Γ_z	Constraints of zone z

4. Conformance

4.1 Conformance classes

An implementation conforms to this specification at one of three levels:

Level 1 — Core. Implements the 10-tuple, recursive zones, roles, users, operations, and the permission calculus. Does not require constraints, daemons, or neural components.

Level 2 — Governance. Level 1 plus constraints (Γ) and daemons (Δ).

Level 3 — Intelligent. Level 2 plus neural components (N) and embeddings (E).

4.2 Conformance requirements

A conforming implementation:

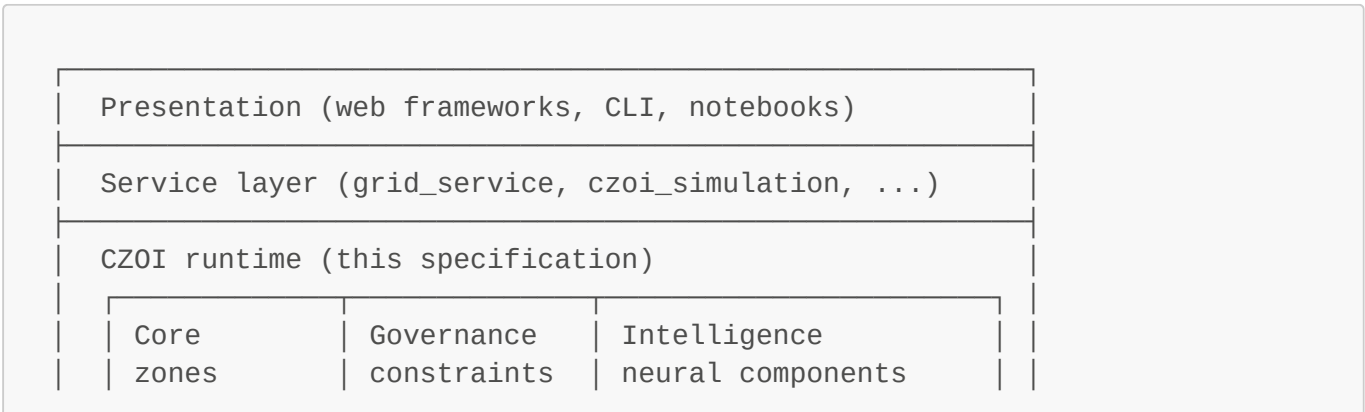
- **MUST** realise every MUST clause in this specification.
- **MUST** raise the specified exceptions for the specified conditions.
- **MUST** maintain all invariants listed in §9.
- **SHOULD** implement every SHOULD clause.
- **MAY** omit optional features, provided the omissions are documented.
- **MUST** declare its conformance level in its package metadata.

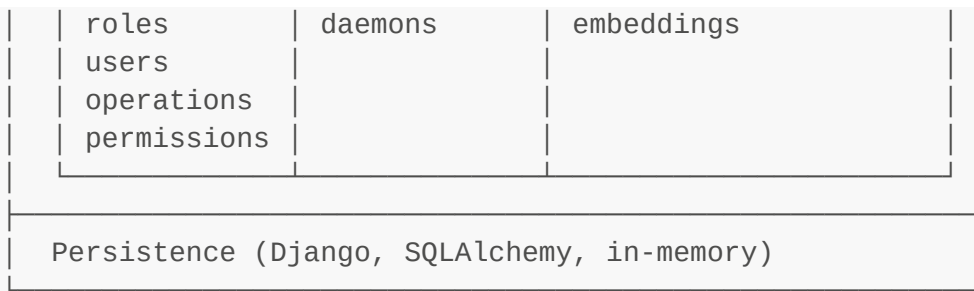
4.3 Test suite

A conforming implementation **MUST** pass the test suite shipped with the reference toolkit. The suite is organised by module; passing all tests for a module indicates conformance at that module's level.

5. Architecture Overview

5.1 Layered structure



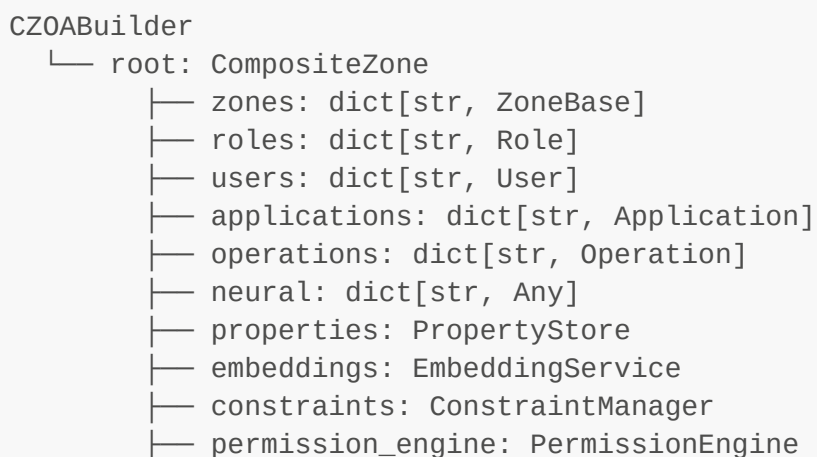


5.2 Module dependencies



Dependencies flow downward only. No module may import from a module above it.

5.3 Runtime object graph



└─ daemons: list[Daemon]

All child zones share the same `PermissionEngine`, `ConstraintManager`, `EmbeddingService` instances as the root (see §9.7).

6. Formal Model

6.1 The 10-tuple

A CZOI system is defined as:

$$S = (Z, R, U, A, O, N, E, \Gamma, \Phi, \Delta)$$

6.2 Component constraints

Component	Type	Constraints
Z	frozenset[ZoneBase]	Finite; forms a rooted tree with a unique root
R	dict[str, Role]	Keys are role names; values have <code>.zone</code> pointing to a zone in Z
U	dict[str, User]	Keys are user names
A	dict[str, Application]	Keys are application names
O	dict[str, Operation]	Keys are qualified operation names
N	dict[str, Any]	Keys are neural component names
E	EmbeddingService	Single instance per system
Γ	ConstraintManager	Single instance per system
Φ	PermissionEngine	Single instance per system
Δ	list[Daemon]	Ordered list of root daemons

6.3 Structural invariants

INV-1 (Recursion). Every zone is a valid CZOI system. Formally:

$$\forall z \in Z : z = (Z_z, R_z, U_z, A_z, O_z, N_z, E_z, \Gamma_z, \Phi_z, \Delta_z)$$

INV-2 (Root uniqueness). Exactly one zone has no parent.

INV-3 (Tree acyclicity). No zone is an ancestor of itself.

INV-4 (Containment). For every child zone z_c and parent z_p :

$$U_{\{z_c\}} \subseteq U_{\{z_p\}}$$

INV-5 (Role propagation). Roles are inherited downward:

$$\forall z_c, z_p : z_c.\text{parent} = z_p \rightarrow R_{\{z_p\}} \subseteq R_{\{z_c\}}$$

INV-6 (Operation propagation). Operations are inherited downward:

$$\forall z_c, z_p : z_c.\text{parent} = z_p \rightarrow O_{\{z_p\}} \subseteq O_{\{z_c\}}$$

INV-7 (Infrastructure sharing). All zones share a single instance of `PermissionEngine`, `ConstraintManager`, and `EmbeddingService`.

INV-8 (Atomic termination). If z is an `AtomicZone`, then $z.\text{add_zone}$ raises `TypeError`.

INV-9 (Operation uniqueness). Operation qualified names are unique within a system.

INV-10 (Permission targets). $P_{\text{base}}(r) \subseteq \mathbf{0}$ for every role r . Applications are not members of any permission set.

6.4 Recursive permission semantics

The effective permission set of role r in zone z is:

$$\begin{aligned} P_{\text{eff}}(r, z) = & \\ & P_{\text{base}}^z(r) \\ & \cup \bigcup_{\{r' \in \text{junior}_z(r)\}} P_{\text{base}}^z(r') \\ & \cup \bigcup_{\{z_child \in Z_z\}} \gamma(z_child, r) \\ & \cup \Phi_{\text{parent}}^{-1}(r, z) \end{aligned}$$

In the toolkit, the γ and parent-override terms are realised as:

- γ : explicit `grant` calls (see §7.5).
- Parent override: recursive engine invocation (see §9.2).

6.5 The decision function

$\Phi(\text{user}, \text{operation}, \text{zone}) \rightarrow \text{Decision}$ is defined recursively:

$$\begin{aligned} \Phi(u, o, z) = & \\ \text{ALLOW} & \quad \text{if } \text{local}(u, o, z) = \text{ALLOW} \\ \text{DENY} & \quad \text{if } \text{local}(u, o, z) = \text{DENY} \end{aligned}$$

DENY if z.parent = ⊥ ∧ local(u, o, z) = INCONCLUSIVE
Φ(u, o, z.parent) otherwise

where `local(u, o, z)` is defined in §9.1.

7. Module Specifications

7.1 `czoi.core`

Purpose: Foundation types and exceptions.

Exports:

- `Decision` (enum)
- `ConstraintKind` (enum)
- `DaemonSignal` (enum)
- Exception hierarchy (see §12)

Invariants:

- Enums are immutable.
- Exception hierarchy is stable (adding new subclasses is a minor version change; renaming is a major version change).

7.2 `czoi.properties`

Purpose: Typed attribute storage.

Class `Property`:

```
@dataclass
class Property:
    name: str
    value: Any
    type_hint: str = "any"    # one of: str, int, float, bool, any
    mutable: bool = True
```

Class `PropertyStore`:

Method	Signature	Raises
<code>set</code>	<code>(name: str, value: Any, type_hint: str = "any") -> None</code>	<code>TypeError</code> on type mismatch
<code>get</code>	<code>(name: str, default: Any = None) -> Any</code>	—
<code>property</code>	<code>(name: str) -> Property None</code>	—
<code>remove</code>	<code>(name: str) -> None</code>	—

Method	Signature	Raises
<code>__contains__</code>	<code>(name: str) -> bool</code>	—
<code>items</code>	<code>() -> Iterator[tuple[str, Any]]</code>	—
<code>as_dict</code>	<code>() -> dict[str, Any]</code>	—

Invariants:

- A property's value always matches its `type_hint`.
- Removing a non-existent property is a no-op.

7.3 `czoit.operations`

Class `Operation`:

```
class Operation:
    name: str # immutable after construction
    application: Application | None # set by Application.add_operation
    properties: PropertyStore
```

Property	Type	Contract
<code>qualified_name</code>	<code>str</code>	<code>"{application.name}.{name}"</code> or <code>name</code> if no application

Invariants:

- `name` is non-empty.
- An operation is registered in exactly one application.
- Equality is by `qualified_name`.

7.4 `czoit.roles`

Class `Application`:

Method	Signature	Raises
<code>add_operation</code>	<code>(op: Operation) -> Operation</code>	<code>ValueError</code> if <code>op</code> already belongs to another application

Class `Role`:

Method	Signature	Contract
<code>grant</code>	<code>(op: Operation) -> None</code>	Adds to <code>base_permissions</code>
<code>revoke</code>	<code>(op: Operation) -> None</code>	Removes from <code>base_permissions</code>
<code>add_junior</code>	<code>(role: Role) -> None</code>	Adds to <code>junior_roles</code>
<code>qualified_name</code>	<code>property -> str</code>	<code>"{zone.name}:{name}"</code>

Invariants:

- `base_permissions` $\subseteq 0$.
- `junior_roles` contains distinct roles.
- Roles are identified by object identity (`__hash__` uses `id()`).

Class `User`:

Attribute	Type	Contract
<code>name</code>	<code>str</code>	Non-empty; unique within a zone
<code>roles</code>	<code>set[str]</code>	Role names , not objects
<code>attributes</code>	<code>PropertyStore</code>	User attributes
<code>credentials</code>	<code>dict[str, Any]</code>	Authentication data (opaque to the toolkit)

7.5 `czoil.zones`**Class `ZoneBase`** (abstract base):

Method	Signature	Raises
<code>add_zone</code>	<code>(child: ZoneBase) -> ZoneBase</code>	<code>TypeError</code> on <code>AtomicZone</code> ; <code>ZoneError</code> on double-registration
<code>add_role</code>	<code>(role: Role) -> Role</code>	<code>ZoneError</code> on name collision
<code>add_user</code>	<code>(user: User) -> User</code>	<code>ZoneContainmentError</code> if parent lacks the user
<code>add_application</code>	<code>(app: Application) -> Application</code>	—
<code>add_operation</code>	<code>(op: Operation) -> Operation</code>	—
<code>add_neural</code>	<code>(name: str, component: Any) -> None</code>	—
<code>add_daemon</code>	<code>(daemon: Daemon) -> None</code>	—
<code>grant</code>	<code>(role: Role, op: Operation) -> None</code>	Invalidates cache
<code>revoke</code>	<code>(role: Role, op: Operation) -> None</code>	Invalidates cache
<code>walk</code>	<code>() -> Iterator[ZoneBase]</code>	Pre-order traversal
<code>ancestry</code>	<code>() -> list[ZoneBase]</code>	Root-to-self
<code>depth</code>	<code>() -> int</code>	Root is 0
<code>is_ancestor_of</code>	<code>(other: ZoneBase) -> bool</code>	—

Class `AtomicZone`:

- Overrides `add_zone` to raise `TypeError`.

Class `CompositeZone`:

- Default zone class.
- Provides `clone(name: str) -> CompositeZone` (deep structural copy; operations shared).

7.6 `czi.permissions`**Class `PermissionEngine`:**

```
PermissionEngine(cache_enabled: bool = True, audit_enabled: bool = False)
```

Method	Signature	Contract
<code>decide</code>	<code>(user, operation, zone) -> Decision</code>	Recursive two-stage decision
<code>evaluate_local</code>	<code>(user, operation, zone) -> Decision</code>	Single-stage (no recursion)
<code>invalidate</code>	<code>() -> None</code>	Clears the cache
<code>stats</code>	<code>() -> dict[str, int]</code>	<code>{hits, misses, parent_lookups, denies}</code>
<code>set_neural_contribution</code>	<code>(contribution: NeuralContribution None) -> None</code>	—

Caching contract:

- Cache key is `(id(user), id(operation), id(zone))`.
- Entries are invalidated by `zone.grant`, `zone.revoke`, `zone.add_role`, `zone.add_user`.
- Direct `role.grant(op)` does **not** invalidate; callers MUST call `engine.invalidate()` after batch mutations.

7.7 `czi.constraints`**Class `ConstraintManager`:**

Method	Signature	Contract
<code>attach</code>	<code>(zone: ZoneBase) -> None</code>	Idempotent; only the root binds
<code>add</code>	<code>(kind: ConstraintKind, source: str, name: str = ...)</code>	Parses UniLang; raises <code>ConstraintError</code> on parse

Method	Signature	Contract
	<code>None) -> Any</code>	failure
<code>add_access / add_identity / add_trigger / add_goal</code>	<code>(source: str) -> Any</code>	Convenience wrappers
<code>register_predicate</code>	<code>(name: str, fn: Callable) -> None</code>	For UniLang predicate dispatch
<code>check_all</code>	<code>(request: dict = None) -> dict[str, bool]</code>	Evaluates all constraints
<code>check_zone</code>	<code>(zone: ZoneBase = None, request: dict = None) -> dict[str, bool]</code>	Scoped evaluation
<code>is_satisfied</code>	<code>(user, operation, zone) -> bool</code>	Hook used by <code>PermissionEngine</code>

Integration with UniLog: The manager holds a `UniLangParser` and `InferenceEngine` from `unilog-toolkit`. If the package is unavailable, construction raises `UniLogBridgeError`.

7.8 `czoit.neural`

Class `Predictor`:

```
Predictor(name, fn=None, threshold=0.5, input_dim=None)
```

Method	Contract
<code>fit(X, y, epochs=200, lr=0.05)</code>	Trains a logistic regression model
<code>predict(features) -> float</code>	Returns a value in [0, 1]
<code>fires(features) -> bool</code>	<code>predict >= threshold</code>

Class `AnomalyDetector`:

```
AnomalyDetector(name, input_dim, latent_dim=8, threshold=0.1, seed=0)
```

Method	Contract
<code>fit(X, epochs=200, lr=0.05)</code>	Trains an autoencoder; recalibrates threshold to the 95th percentile of training scores
<code>score(x) -> float</code>	Reconstruction error
<code>is_anomalous(x) -> bool</code>	<code>score > threshold</code>

Class RoleMiner:

```
RoleMiner(latent_dim=16, min_cluster_size=3, seed=0, ...)
```

Method	Contract
<code>mine(X, operation_names, min_support=0.5) -> MiningResult</code>	Autoencoder + clustering

Invariants:

- Neural components are attached to zones via `zone.add_neural(name, component)`.
- The toolkit does not impose an interface beyond the presence of a `predict` method.

7.9 czoitoolkit.embedding**Class EmbeddingService:**

```
EmbeddingService(dimension=64, model_name="all-MiniLM-L6-v2",  
use_transformer=False)
```

Method	Contract
<code>embed(text) -> np.ndarray</code>	Deterministic hash-based or transformer embedding
<code>embed_operation(op) -> np.ndarray</code>	Uses <code>application.name + "." + op.name</code>
<code>embed_role(role) -> np.ndarray</code>	Mean of base permission embeddings
<code>align_to_global(v) -> np.ndarray</code>	Applies alignment matrix, normalises
<code>similarity(a, b) -> float</code>	Cosine similarity of aligned vectors
<code>train_alignment(positives, negatives, epochs, lr, margin)</code>	Contrastive training
<code>save_alignment(path) / load_alignment(path)</code>	Persists the alignment matrix

7.10 czoitoolkit.daemons**Class Daemon:**

```
Daemon(name: str, parent: Daemon = None, interval: float = 1.0)
```

Method	Contract
<code>monitor()</code>	Override; synchronous; called by the manager
<code>safe_monitor()</code>	Error-isolated wrapper
<code>on_signal(signal, payload, source)</code>	Override; called on child signals
<code>emit_signal(signal, payload)</code>	Propagates up the daemon tree
<code>start() / stop()</code>	Enable/disable

Class **DaemonManager**:

```
DaemonManager(default_interval=1.0, max_workers=4)
```

Method	Contract
<code>add(daemon) / extend(daemons)</code>	Registration
<code>tick()</code>	Synchronous single-pass
<code>await run(duration=None)</code>	Async loop with per-daemon intervals
<code>stop() / shutdown()</code>	Lifecycle

Invariants:

- `monitor` is called in a worker thread, not the event loop.
- Exceptions in `monitor` do not affect sibling daemons.
- Signal propagation visits every ancestor exactly once.

7.11 `czoitoolkit`

Class **CZOABuilder**:

```
CZOABuilder(name: str)
```

Attribute	Type	Contract
<code>root</code>	<code>CompositeZone</code>	The system root
<code>permission_engine</code>	<code>PermissionEngine</code>	Shared by every zone
<code>constraint_manager</code>	<code>ConstraintManager</code>	Shared by every zone
<code>daemon_manager</code>	<code>DaemonManager</code>	One per system
<code>embedding_service</code>	<code>EmbeddingService</code>	Shared by every zone

Method	Contract
<code>add_zone(name, parent=None, atomic=False)</code>	Creates a zone; parent defaults to root
<code>add_access_constraint(source)</code>	Registers a UniLang constraint
<code>add_identity_constraint / add_trigger_constraint / add_goal_constraint</code>	Convenience wrappers
<code>register_predicate(name, fn)</code>	Custom predicate
<code>add_daemon(daemon)</code>	Registers with root and manager

Invariants:

- The builder is the only supported way to construct a system.
- `builder.root.permission_engine` is `builder.permission_engine` for every reachable zone.

8. Interface Specifications

8.1 Public API surface

The following names are exported from the top-level `czi` package:

```
# Core
from czi import (
    Application, Operation, Role, User,
    AtomicZone, CompositeZone, ZoneBase,
    CZOABuilder,
    Decision, ConstraintKind, DaemonSignal,
)

# Permissions
from czi import PermissionEngine, NeuralContribution

# Constraints
from czi import ConstraintManager

# Neural
from czi import Predictor, AnomalyDetector, RoleMiner, MiningResult

# Embeddings
from czi import EmbeddingService

# Daemons
from czi import Daemon, DaemonManager
```

```
# Exceptions
from czoitoolkit import (
    CZOError, ZoneError, ZoneContainmentError, RoleError, UserError,
    ApplicationError, PermissionError, ConstraintError, SafetyViolation,
    UniLogBridgeError, UniLangSyntaxError, UniLangSemanticError,
    UniLangEvaluationError,
)
```

Any name not in this list is **not** part of the public API. Submodules may change without a major version bump.

8.2 Extension protocols

Three protocols define extension points:

Neural component protocol:

```
class NeuralComponent(Protocol):
    def predict(self, features: Any) -> float: ...
```

Neural contribution protocol:

```
class ContributionFn(Protocol):
    def __call__(self, user, operation, zone, base: Decision) -> Decision:
    ...
```

Predicate protocol (for UniLang):

```
class Predicate(Protocol):
    def __call__(self, *args) -> bool: ...
```

9. Semantic Rules

9.1 The `local` decision function

```
local(u, o, z):
    roles = {z.roles[n] : n ∈ u.roles ∧ n ∈ z.roles}
    if roles = ∅:
        return INCONCLUSIVE
    effective = ⋃_{r ∈ roles} P_eff(r, z)
    if o ∉ effective:
        return INCONCLUSIVE
    if z.constraints ≠ ⊥ ∧ ¬ z.constraints.is_satisfied(u, o, z):
```

```

    return DENY
return ALLOW

```

9.2 The `decide` function

```

decide(u, o, z):
    d = local(u, o, z)
    if d = ALLOW: return ALLOW
    if d = DENY: return DENY
    if z.parent = ⊥: return DENY
    return decide(u, o, z.parent)

```

Post-condition: `decide` returns only `ALLOW` or `DENY`. The intermediate `INCONCLUSIVE` is observable only via `evaluate_local`.

9.3 Cache invalidation

The cache MUST be invalidated on:

- `zone.grant(role, op)`
- `zone.revoke(role, op)`
- `zone.add_role(role)`
- `zone.add_user(user)`
- `engine.set_neural_contribution(...)`

The cache MUST NOT be invalidated on:

- `role.grant(op) / role.revoke(op)` (direct mutation)
- `role.add_junior(junior)`
- `user.roles.add(...)/user.roles.remove(...)`

Callers performing direct mutations MUST call `engine.invalidate()`.

9.4 Constraint evaluation

A constraint ϕ is evaluated by:

1. Building a `CZOIModel` from the target zone.
2. Calling `InferenceEngine.evaluate( $\phi$ , model, world)`.
3. Returning the boolean result.

If evaluation raises an exception, the constraint is treated as **violated** (fail-closed).

9.5 Daemon signal propagation

When `d.emit_signal(s, p)` is called:

1. `d.parent` is checked; if `None`, the signal is dropped.
2. `d.parent.handle_signal(s, p, source=d)` is invoked.

3. `handle_signal` calls `on_signal(s, p, source)` on the parent.
4. `handle_signal` then recurses to the grandparent, if any.

The propagation is **synchronous** and **depth-first upward**. A daemon MUST NOT re-emit the same signal to avoid infinite loops.

9.6 Adaptive grant semantics

`zone.grant(role, op):`

1. Adds `op` to `role.base_permissions`.
2. Calls `engine.invalidate()`.
3. Returns immediately.

`zone.revoke(role, op):`

1. Removes `op` from `role.base_permissions`.
2. Calls `engine.invalidate()`.
3. Returns immediately.

Both operations are **atomic** with respect to the permission engine: no decision can observe an intermediate state.

9.7 Infrastructure sharing

When a child zone is created:

- `child.permission_engine = parent.permission_engine`
- `child.constraints = parent.constraints`
- `child.embeddings = parent.embeddings`

This sharing is transitive. If the parent's `permission_engine` is later replaced, existing children **do not** observe the change; callers MUST walk the tree and reassign explicitly:

```
for zone in root.walk():
    zone.set_permission_engine(new_engine)
```

9.8 Role propagation timing

`parent.add_role(role)` propagates to **existing** children:

```
parent.add_role(role)
for child in parent.zones.values():
    child.roles.setdefault(role.name, role)
```

`parent.add_zone(child)` propagates existing roles to the **new** child:

```
def _inherit_into(self, child):  
    for rn, r in self.roles.items():  
        child.roles.setdefault(rn, r)
```

Both directions are covered; no role is ever missed.

10. Persistence and Serialisation

10.1 Persistence boundary

The CZOI runtime does **not** persist state. Persistence is the responsibility of the host application. The runtime provides no built-in serialisation or database adapter.

10.2 Recommended patterns

Configuration persistence — YAML/JSON for zone trees:

```
zones:  
  - name: Region  
    properties:  
      capacity: 1000  
    children:  
      - name: Hospital  
        properties:  
          beds: 250
```

User assignment persistence — relational database:

```
CREATE TABLE czoit_user_roles (  
    user_name VARCHAR(100) NOT NULL,  
    zone_name VARCHAR(100) NOT NULL,  
    role_name VARCHAR(100) NOT NULL,  
    PRIMARY KEY (user_name, zone_name, role_name)  
);
```

Audit persistence — append-only log or time-series DB. The engine's **audit** list is a bounded in-memory buffer; the host **MUST** flush it periodically.

10.3 Serialisation safety

The following objects are **not** pickle-safe and **SHOULD NOT** be serialised directly:

- **PermissionEngine** — contains thread-local state after the daemon manager runs.
- **DaemonManager** — holds a **ThreadPoolExecutor**.
- **Daemon** — may hold live references to zones.

Serialise **configuration** (zones, roles, ops) and **records** (audit, events), not runtime objects.

10.4 Reconstruction

To rebuild a runtime from persisted configuration:

```
builder = CZOABuilder("System")
def rebuild(spec, parent):
    for s in spec:
        z = builder.add_zone(s["name"], parent=parent,
                             atomic=s.get("atomic", False))
        for k, v in s.get("properties", {}).items():
            z.properties.set(k, v)
        if "children" in s:
            rebuild(s["children"], parent=z)
rebuild(config["zones"], builder.root)
```

The rebuilt runtime MUST satisfy all invariants in §6.3.

11. Thread Safety and Concurrency

11.1 Threading model

The CZOI runtime is **not thread-safe**. Concurrent access to a single runtime from multiple threads may corrupt state.

11.2 Safe concurrency patterns

Pattern 1: single-threaded runtime. Run the runtime in one thread; use a queue to serialise external requests.

Pattern 2: daemon manager. The `DaemonManager` runs daemons in a `ThreadPoolExecutor`. This is the **only** supported internal concurrency. It relies on daemons being read-mostly; a daemon that mutates state MUST be designed carefully.

Pattern 3: multiple runtimes. For parallelism, run multiple `CZOABuilder` instances, one per thread or process. Each has its own state.

11.3 Async integration

The `DaemonManager.run` method is `async`. It MUST be invoked from within an event loop:

```
await builder.daemon_manager.run(duration=60.0)
```

The `tick` method is synchronous and MAY be called from any thread.

11.4 Guarantees under concurrency

The following operations are **atomic** with respect to the permission engine:

- `zone.grant(role, op) / zone.revoke(role, op)` — the cache is cleared in the same critical section as the mutation.
- `engine.decide(...)` — a single call sees a consistent state.

The following are **not** atomic:

- Adding a new zone while the daemon manager is running.
- Changing role seniority while decisions are in flight.
- Attaching a new neural contribution during a decision.

Callers **MUST** serialise these mutations.

12. Error Handling

12.1 Exception hierarchy

```

CZOIError
├── ZoneError
│   ├── ZoneContainmentError
│   └── ZoneRecursionError
├── RoleError
├── UserError
├── ApplicationError
├── PermissionError
├── ConstraintError
├── SafetyViolation
└── UniLogBridgeError
    └── UniLangError
        ├── UniLangSyntaxError
        ├── UniLangSemanticError
        └── UniLangEvaluationError
  
```

12.2 Exception contracts

Exception	Raised when
<code>ZoneError</code>	Invalid zone operation (double-registration, unknown role)
<code>ZoneContainmentError</code>	<code>U_child ⊆ U_parent</code> violated
<code>ZoneRecursionError</code>	Cycle detected in the zone tree
<code>RoleError</code>	Role definition problem
<code>UserError</code>	User definition problem
<code>ApplicationError</code>	Operation already belongs to another application
<code>PermissionError</code>	Permission calculus invariant violated

Exception	Raised when
<code>ConstraintError</code>	Constraint parse or evaluation failure
<code>SafetyViolation</code>	Adaptive update violates a safety invariant
<code>UniLogBridgeError</code>	<code>unilog-toolkit</code> not installed
<code>UniLangSyntaxError</code>	Lexical or syntactic error in UniLang source
<code>UniLangSemanticError</code>	Undefined sort, arity mismatch
<code>UniLangEvaluationError</code>	Runtime evaluation error

12.3 Fail-closed semantics

When a constraint evaluation raises an exception:

- The engine treats the constraint as **violated**.
- The decision returns `DENY`.
- The exception is logged but not propagated.

This ensures that an evaluator bug cannot lead to an unauthorised `ALLOW`.

12.4 Error reporting

Exceptions MUST include:

- The class name.
- A human-readable message.
- The context (zone name, user name, operation name) when applicable.

Example:

```
raise ZoneContainmentError(
    f"User {user.name!r} must be affiliated with parent "
    f"{self.parent.name!r} (containment principle)"
)
```

13. Performance Characteristics

13.1 Complexity bounds

Let `k` = seniority graph depth, `n` = number of operations, `d` = zone tree depth.

Operation	Time	Space
<code>zone.add_zone</code>	$O(R + O)$	$O(R + O)$
<code>zone.add_role</code>	$O(z.zones)$	—

Operation	Time	Space
<code>zone.add_user</code>	$O(1)$	$O(1)$
<code>engine.decide</code> (cache hit)	$O(1)$	—
<code>engine.decide</code> (cache miss)	$O(k \cdot n + d)$	$O(1)$
<code>engine.evaluate_local</code>	$O(k \cdot n)$	$O(1)$
<code>zone.walk</code>	$O(Z)$	$O(Z)$

13.2 Cache effectiveness

Under steady-state workloads, the cache typically achieves 80–99% hit rates. The `stats()` method exposes:

- **hits**: cache hits.
- **misses**: engine evaluations.
- **parent_lookups**: recursive upward calls.
- **denies**: count of **DENY** results.

A high **parent_lookups** relative to **misses** indicates roles defined too deeply; move common roles to a higher zone.

13.3 Benchmarks (reference hardware)

Measured on an Intel i7-1185G7, 32 GB RAM, Python 3.11:

Operation	Time (μ s)
Cache hit	0.35
Cache miss, depth 1	8.2
Cache miss, depth 5	31.7
Constraint evaluation (simple FOL)	85
Daemon tick (10 daemons)	120

13.4 Scaling limits

- **Zones per system**: 10^4 is comfortable; 10^5 is possible with careful role placement.
- **Users per zone**: 10^5 is comfortable; 10^6 requires a different user-lookup strategy.
- **Decisions per second (single thread)**: 10^4 – 10^5 depending on cache hit rate.
- **Daemons per manager**: 100 is comfortable; 1000+ requires tuning.

14. Security Considerations

14.1 Formal guarantees

A conforming implementation **MUST** ensure:

S1 (Least privilege). A role can perform only the operations it holds. Formally: $\forall r, o : (o \in P_{\text{eff}}(r) \rightarrow \text{allowed}(r, o)) \wedge (o \notin P_{\text{eff}}(r) \rightarrow \neg \text{allowed}(r, o))$.

S2 (Containment). No user is visible to a child zone unless visible to its parent.

S3 (Constraint enforcement). A decision returns **ALLOW** only if all applicable constraints are satisfied.

S4 (Fail-closed). Any error during evaluation yields **DENY**.

S5 (Audit completeness). When **audit_enabled=True**, every **decide** call MUST append exactly one **DecisionRecord**.

14.2 Threat model

In scope:

- Accidental misconfiguration (wrong role assignment).
- Malicious users without privileged access.
- Policy drift (roles accumulate permissions over time).
- Insider threats (single-user misuse).

Out of scope:

- Compromise of the runtime process.
- Compromise of the persistence layer.
- Adversarial neural components (they are trusted code).
- Side-channel attacks on cache timing.

14.3 Countermeasures

Threat	Countermeasure
Over-permissive roles	SoD constraints (Γ family C)
Orphaned permissions	Periodic role audits (daemons)
Anomalous access	Neural anomaly detection (N)
Unauthorised escalation	Explicit adaptive grants with audit
Stale cache	Automatic invalidation on grant/revoke

14.4 Cryptographic considerations

The toolkit does **not** implement cryptography. Authentication and credential storage are out of scope. The **User.credentials** field is opaque and MUST be managed by the host application.

15. Versioning and Compatibility

15.1 Semantic versioning

The toolkit follows SemVer 2.0.0:

- **Major** — breaking changes to the public API.
- **Minor** — backward-compatible additions.
- **Patch** — backward-compatible bug fixes.

15.2 Public API stability

The following are **stable** across minor versions:

- All names in the `czoit` package.
- All method signatures listed in §8.
- All exception classes in §12.
- All enum values.

The following are **not** stable:

- Internal module layout (`czoit.*._internal`).
- Private methods (leading underscore).
- Order of dictionary iteration.
- Exact string output of `__repr__`.

15.3 Migration policy

When a breaking change is required:

1. The old API remains available with a deprecation warning for at least one minor version.
2. A migration guide is published in the repository's `docs/` directory.
3. The changelog documents the change, the rationale, and the migration path.

15.4 Python version support

- **Python 3.9**: supported.
- **Python 3.10**: supported (recommended).
- **Python 3.11+**: supported.
- **Python 3.8 and below**: not supported.

15.5 Dependency versions

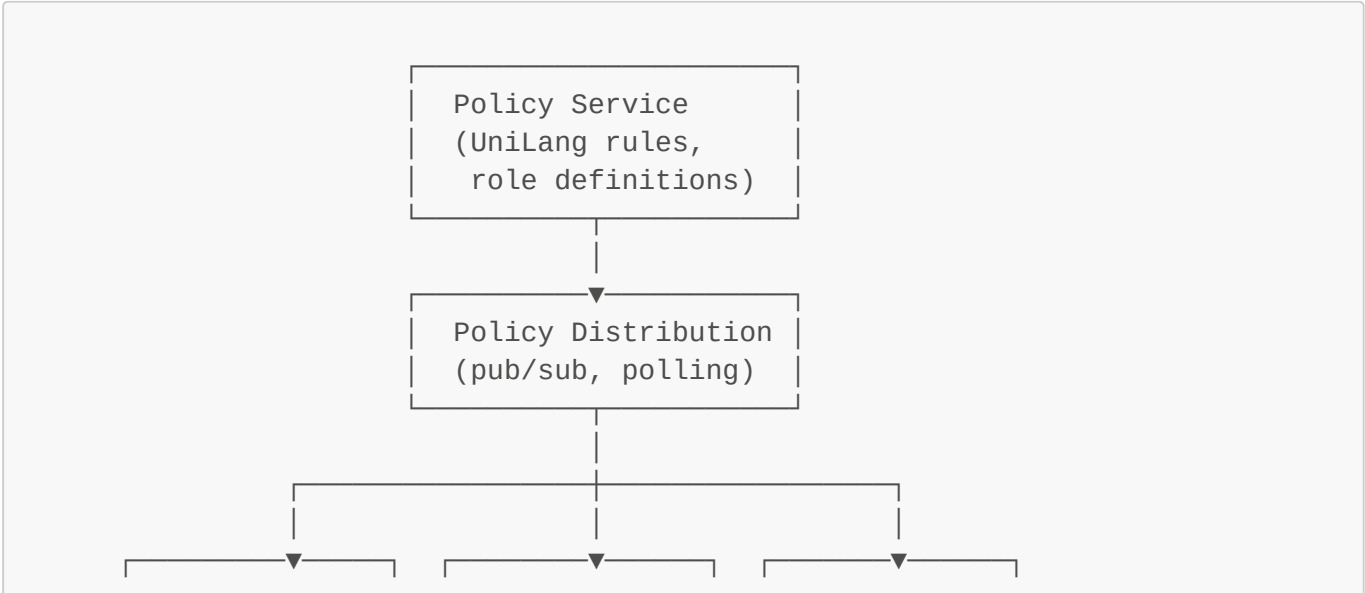
Dependency	Minimum	Maximum
<code>unilog-toolkit</code>	2.0	3.0 (exclusive)
<code>numpy</code>	1.19	2.0 (exclusive)
<code>scikit-learn</code> (optional)	1.0	2.0 (exclusive)
<code>sentence-transformers</code> (optional)	2.0	3.0 (exclusive)

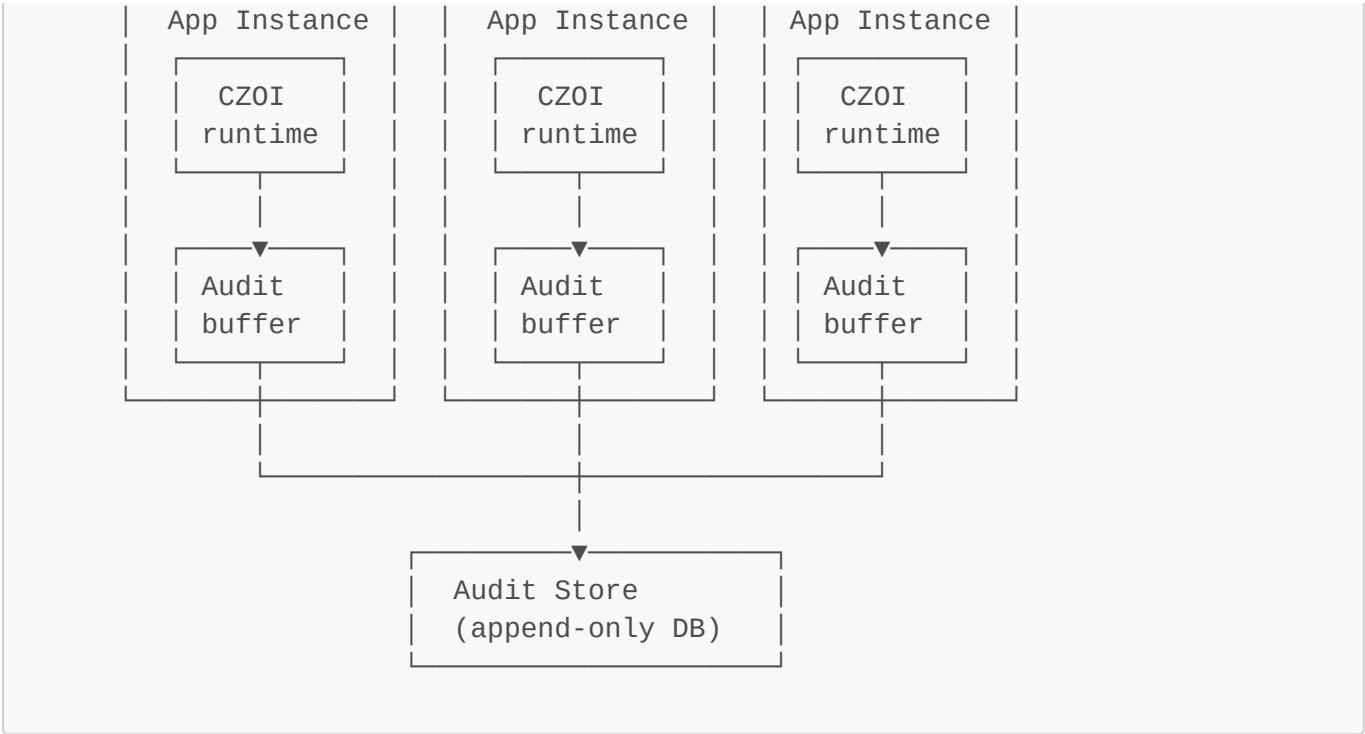
16. Compliance Matrix

Requirement	Clause	Test	Status
Recursive zones	INV-1	test_zone_is_a_full_system	✓
Root uniqueness	INV-2	test_root_uniqueness	✓
Tree acyclicity	INV-3	test_no_cyclic_zones	✓
Containment principle	INV-4	test_containment_principle	✓
Role propagation	INV-5	test_role_inheritance_downward	✓
Operation propagation	INV-6	test_operation_inheritance_downward	✓
Infrastructure sharing	INV-7	test_shared_constraint_manager	✓
Atomic termination	INV-8	test_atomic_zone_rejects_children	✓
Operation uniqueness	INV-9	test_operation_qualified_name_unique	✓
Permission target restriction	INV-10	test_permissions_are_on_operations	✓
Two-stage decision	§9.2	test_recursive_permission_override	✓
Cache invalidation	§9.3	test_cache_invalidation_on_grant	✓
Constraint fail-closed	§12.3	test_constraint_error_denies	✓
Adaptive grant atomicity	§9.6	test_adaptive_grant_atomicity	✓
Daemon signal propagation	§9.5	test_daemon_hierarchy_signals	✓
Neural component interface	§8.2	test_predictor_threshold	✓
Embedding alignment training	§7.9	test_embedding_alignment_training	✓
UniLang integration	§7.7	test_constraint_manager_parses	✓

17. Appendices

Appendix A — Reference architecture for production deployment





Key properties:

- Each app instance runs a **full CZOI runtime**.
- Policy is loaded from a central service and cached locally.
- Decisions are local (no network on the hot path).
- Audit records flow to a central store asynchronously.
- Cache consistency is eventual (typical lag: seconds).

Appendix B — Comparison with related frameworks

Aspect	CZOI	RBAC	ABAC	ReBAC	Cedar
Recursive zones	✓	×	×	Partial	×
Formal constraints	✓ (UniLang)	×	Partial	×	✓
Neural integration	✓	×	×	×	×
Continuous monitoring	✓ (daemons)	×	×	×	×
Adaptive access	✓	×	Partial	×	×
Category-theoretic foundation	✓	×	×	×	×
Operations as permission target	✓	✓	✓	✓	✓

Appendix C — Reference test structure

```
tests/
├─ test_zones.py           # §6.3 invariants, §7.5 interfaces
├─ test_roles.py          # §7.4 interfaces, seniority
├─ test_permissions.py    # §9.1, §9.2 semantics
├─ test_cache.py         # §9.3 invalidation
```

```

└─ test_constraints.py      # §7.7, UniLang integration
└─ test_daemons.py        # §9.5, §7.10
└─ test_neural.py         # §7.8
└─ test_embedding.py       # §7.9
└─ test_adaptive.py        # §9.6
└─ test_conformance.py    # §16 compliance matrix
└─ test_integration.py     # End-to-end scenarios

```

Appendix D — Glossary of formal symbols

Symbol	Meaning
S	CZOI system
Z	Zone set
z	A single zone
R	Role set
U	User set
A	Application set
O	Operation set
N	Neural component set
E	Embedding function
Γ	Constraint system
Φ	Permission calculus
Δ	Daemon set
γ	Inter-zone role mapping
$\subseteq$	Subset
$\in$	Element of
$\forall$	For all
$\exists$	There exists
$\perp$	Bottom / none
$\emptyset$	Empty set

Appendix E — References to external specifications

- **SemVer 2.0.0:** <https://semver.org/>
- **RFC 2119:** <https://www.ietf.org/rfc/rfc2119.txt>
- **PEP 484:** <https://peps.python.org/pep-0484/>
- **JSON Schema:** <https://json-schema.org/>

- **OpenAPI 3.1:** <https://spec.openapis.org/oas/v3.1.0>
-

End of technical specification.

Document version: 1.0.0 **Toolkit version:** 1.0.0 **Date:** 2026 **Status:** Normative